CS 2006 — Operating Systems

Spring 2026

Chrono Rift

Project Report

Group Members

Member A: Fatima Hashmat – 24I-0526

Member B: Haleema Sadia – 24I-0654

NUCES — FAST University

Submission Date: 10 May 2026

1. Project Overview

Chrono Rift is a multi-process, turn-based tactical battle game developed as part of the CS 2006 Operating Systems course. The game demonstrates core OS concepts including inter-process communication (IPC) via shared memory, multi-threading with POSIX pthreads, semaphore-based synchronisation, signal handling, and process scheduling.

The system is divided into three separate processes:

- **Arbiter** — Central authority managing the global game state, scheduling, and rule enforcement.
- **HIP (Human Interfacing Process)** — Captures player input via SFML and forwards actions through shared memory.
- **ASP (Automated Strategic Process)** — Manages enemy AI; each NPC runs in its own dedicated thread.

All inter-process communication is implemented exclusively through POSIX shared memory and unnamed semaphores. The use of pipes is strictly prohibited per the project specification and was not used anywhere in the implementation.

2. Turnaround Time Analysis

2.1 Scheduling Model

Chrono Rift uses a stamina-based temporal scheduling model. Every entity — player or enemy — accumulates stamina each tick at a rate equal to its speed attribute. An entity becomes eligible to act only when its stamina reaches the maximum threshold. Once an action is committed, the entity's stamina is reset to zero and the scheduler recalculates the next actor.

2.2 Turnaround Time Formula

The first-turn turnaround time for any entity is calculated as:

$$\text{Turnaround Time} = \text{Max Stamina} \div \text{Speed} \quad \text{(seconds)}$$

The parameters for this game, seeded by Roll Number 5265, are:

- Player Max Stamina: 100 | Player Speed: 100 / number of players in party
- Enemy Max Stamina: 150 | Enemy Speed: random between 10 and 30 per run

2.3 Turnaround Time Table

The table below shows calculated first-turn turnaround times for representative configurations:

Entity	Max Stamina	Speed	Turnaround Time
Player (1-party)	100	100	1.0 sec
Player (2-party)	100	50	2.0 sec
Player (4-party)	100	25	4.0 sec

Enemy (fast, spd=30)	150	30	5.0 sec
Enemy (mid, spd=20)	150	20	7.5 sec
Enemy (slow, spd=10)	150	10	15.0 sec

As shown, a solo player acting at full speed (100) takes exactly 1 second per turn, whereas a 4-player party has each character waiting 4 seconds. Enemy turnaround ranges from 5 seconds (fastest, speed 30) to 15 seconds (slowest, speed 10).

2.4 Fairness via Normalised Stamina

A key design decision in the scheduler is the use of normalised stamina fractions rather than raw stamina values when comparing entities. The scheduler selects the next actor using the expression:

```
fraction = current_stamina / max_stamina
```

This is critical because players and enemies have different maximum stamina values (100 vs 150). Without normalisation, an enemy at 130 stamina would always be selected over a player at 100 stamina, even though the player is fully charged and the enemy is not. By comparing fractions, a fully charged player (fraction = 1.0) correctly beats a partially charged enemy (fraction = 0.87), ensuring fair scheduling that is proportional to each entity's own stamina cycle.

2.5 Wave Respawn and Scheduling Continuity

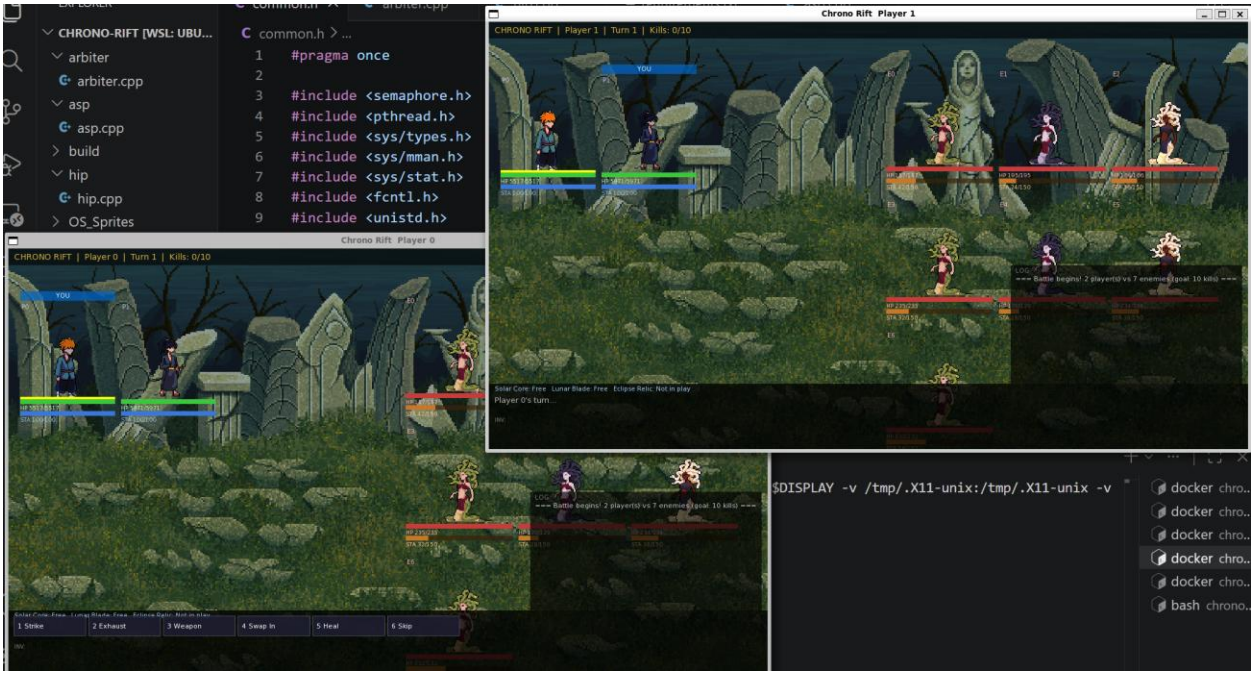
The game implements a wave-based enemy respawn system. The win condition requires eliminating a total of 10 enemies across all waves. When all enemies in the current wave are defeated, the Arbiter spawns a new batch (2–9 enemies, capped at the remaining kill quota). The stamina tick continues uninterrupted across waves, so player stamina is preserved and scheduling resumes immediately with the new enemies starting at zero stamina.

3. Gameplay Screenshots

The following screenshots were captured during live gameplay inside the Docker environment on Ubuntu 22.04. Each screenshot is annotated with a brief description of the OS concept it demonstrates.

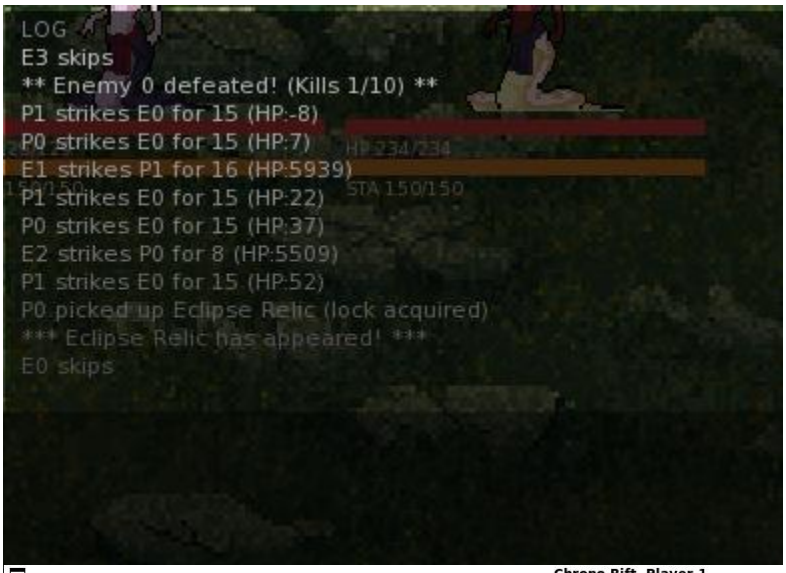
3.1 Game Running — Dual HIP Windows

Both HIP windows (Player 0 and Player 1) are visible simultaneously, each running as a separate process with its own SFML window. The title bar shows the current turn number and kill count. Player sprites, enemy sprites, HP bars, stamina bars, and the action log are all rendered in real time from shared memory.



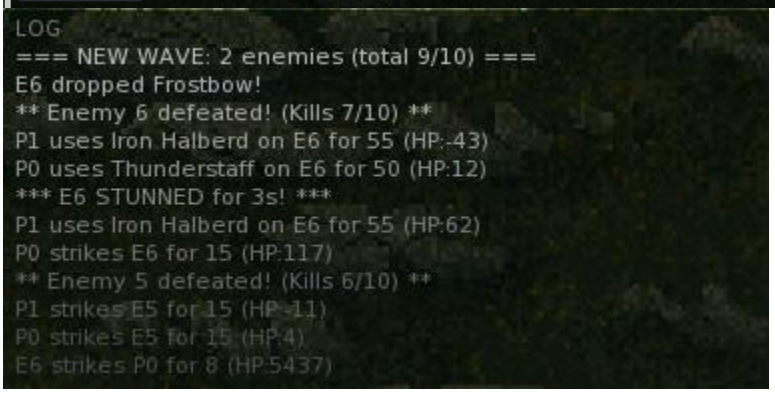
3.2 Kill Registered in Action Log

The action log panel (bottom-right) shows a kill event — e.g. 'Enemy 2 defeated! (Kills 3/10)'. This confirms the Arbiter correctly updates the shared kill counter and broadcasts it to both HIP windows via shared memory.



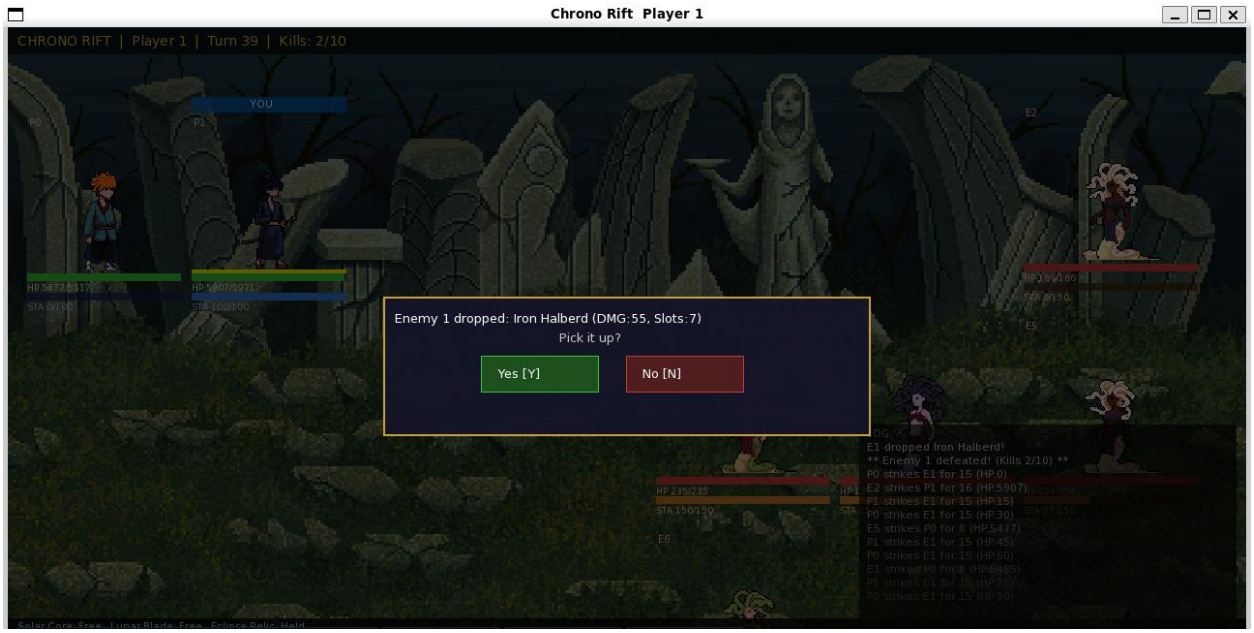
3.3 Wave 2 Enemy Respawn

After all enemies in Wave 1 are eliminated, Wave 2 spawns automatically. The action log shows '=== NEW WAVE: 5 enemies (total 10/10) ===='. New enemy sprites appear on the right side of the screen in their idle animation state. This demonstrates the Arbiter's wave management and ASP thread re-spawning logic.



3.4 Weapon Drop and Pickup Prompt

When an enemy is defeated without holding a weapon, there is a 50% chance of a weapon drop. A modal overlay appears mid-screen prompting the active player to accept or decline the weapon, showing the weapon name, damage value, and inventory slot size. If declined, the weapon is automatically assigned to a random living enemy.



3.5 Game Over / Victory Screen

The win screen appears when all 10 enemies across all waves have been defeated. A full-screen overlay displays 'VICTORY!' in yellow, along with the final kill count (10/10). The loss screen shows 'GAME OVER' in red when all player characters reach zero HP.


VICTORY!
KILLS: 10 / 10

4. OS Concepts Implemented

The table below summarises the key OS concepts demonstrated in Chrono Rift:

Concept	Implementation in Chrono Rift
Shared Memory (IPC)	All game state held in a single POSIX shm segment (/chrono_rift_shm), accessible by all three processes.
Unnamed Semaphores	Per-player and per-enemy semaphores coordinate turn signalling without pipes.
Pthreads	Each NPC runs its own thread in ASP; HIP has separate render and input threads.
Process-Shared Mutexes	state_mutex, artifact_mutex, and action_log.mutex use PTHREAD_PROCESS_SHARED.
Signal Handling	SIGTERM triggers graceful quit; SIGUSR1 delivers stun; SIGSTOP/SIGCONT pause ASP for Ultimate.
Temporal Scheduling	Stamina-based scheduler with normalised fraction comparison for fairness.
Deadlock Detection	Background thread in Arbiter monitors circular waits on artifacts and forces release.
Wave Management	Arbiter respawns enemy batches via asp_wave_ready flag; ASP polls and re-spawns threads.

This project was compiled and tested inside the provided Docker environment (Ubuntu 22.04) using the SFML GUI library.